

Visualización y analítica de datos masivos

Práctica Informática – Entrega Intermedia

ALUMNO: RAQUEL ABREU HERNANDO

Temática Objetivo de la visualización: Análisis del impacto del ruido en la vida de las personas.

1. Introducción de la entrega intermedia

Partiendo de la entrega inicial del caso práctico, previamente validada por el equipo docente, este documento constituye la **entrega intermedia** de la práctica informática. Su propósito es presentar el proceso seguido para la **preparación y depuración del conjunto de datos** que servirá de base para la visualización que se entregará en la entrega final de la práctica.

En esta fase del trabajo se incluyen los siguientes elementos:

- **Resultado del proceso de preparación de los datos.** Se detalla en este documento la metodología seguida y se proporciona el enlace al repositorio **GitHub** donde se estructura el proyecto completo: <https://github.com/raquelabreu/Ruido-Visualizacion>
- **Cuadernos Jupyter** empleados para la limpieza, transformación y preparación de los datos. Dichos cuadernos están disponibles tanto en GitHub como en la plataforma de la UNED.
- Dado el **gran volumen del conjunto de datos**, se ha habilitado un **repositorio en la nube** a través de **Kaggle**, accesible para el equipo docente. Se proporcionan los enlaces correspondientes:
 - **Repositorio con los datos originales:**
<https://www.kaggle.com/datasets/raquelahdo/ruido-datasets>
 - **Repositorio con los datos procesados:**
<https://www.kaggle.com/datasets/raquelahdo/ruido-datasets-procesados>

Además de los enlaces anteriores, este documento describe de forma detallada el **proceso completo de preparación de los datos**, desde su obtención y análisis preliminar hasta su transformación final para garantizar su idoneidad en la posterior fase de visualización.

Adicionalmente, se ha creado un repositorio en la nube OneDrive (cuenta del estudiante UNED) donde quedan disponibles los datos para el equipo docente en los siguientes links:

- Datos originales: [ruido-datasets-raw data.zip](#)
- Datos procesados: [df_datos-procesados.zip](#)
- [Jupyter notebooks- fase preprocesado.zip](#)

2. Resultado del proceso de preparación de los datos

El proceso de preparación de los datos constituye la base del proyecto de visualización y asegura la reproducibilidad, trazabilidad y correcta organización del flujo de trabajo. Para ello, se ha estructurado un entorno de trabajo apoyado en herramientas web que permiten gestionar el código, la documentación y los conjuntos de datos de forma eficiente.

- **Uso de GitHub**

En primer lugar, **GitHub** se ha utilizado como repositorio central del proyecto. En él se alojan los **cuadernos Jupyter** empleados para las fases de limpieza, normalización, exploración y visualización, así como la documentación asociada (instrucciones de ejecución, memoria y elementos exportados como la visualización en HTML). Esta estructura garantiza un control de versiones completo, una clara separación entre código y datos, y una guía de ejecución reproducible para el equipo docente.

El repositorio incluye además la estructura estandarizada de proyecto —con carpetas para datos en bruto (data/raw), datos procesados (data/processed) y scripts del pipeline— junto con un archivo requirements.txt que facilita la instalación de dependencias, que en esta fase del proyecto no se utiliza, pero queda disponible para la entrega final.

Repositorio del proyecto: <https://github.com/raq-abreu/Ruido-Visualizacion>

- **Uso de Kaggle**

Por otro lado, debido al tamaño de los ficheros de datos (superior a 50 MB), y siguiendo buenas prácticas de gestión de proyectos y políticas de GitHub, los datasets no se almacenan en el repositorio. En su lugar, se utiliza **Kaggle** como plataforma de alojamiento de datos voluminosos. Esta elección se ajusta a las indicaciones del enunciado de la práctica, que recomienda emplear un repositorio externo cuando el volumen de datos impide su carga directa en la plataforma docente.

En Kaggle se publican tanto los conjuntos de datos originales como los datasets ya procesados, utilizados posteriormente en las fases de análisis y visualización. Esta separación permite un acceso eficiente, estable y centralizado a todos los ficheros necesarios para la ejecución del proyecto por parte del equipo docente.

Kaggle permite gestionar datasets voluminosos con **versionado automático**, trazabilidad de cambios y acceso mediante enlaces privados o públicos.

En conjunto, esta organización garantiza que el proceso de preparación de los datos sea transparente, auditable y fácilmente reproducible, además de permitir la correcta trazabilidad de cada etapa desde la obtención de los datos hasta su transformación final para la visualización.

2.1. Kaggle – Repositorio con los datos originales

El repositorio utilizado para los datos originales es el siguiente:

Kaggle Dataset (datos originales):<https://www.kaggle.com/datasets/raquelahdo/ruido-datasets>

Tamaño total: 573.48 MB

Número de ficheros: 10

Los ficheros incluidos en la versión 3 del dataset con el que se han preparado los datos son los siguientes:

1. [01_Ruido_diario_acumulado.csv](#) (23.18MB). Datos de ruido diario acumulado por zona. 11 columnas y aproximadamente 521.000 filas.
2. [02_contaminacion-acustica.csv](#) (694.39kB). Series temporales relacionadas con contaminación acústica. 13 columnas.
3. [03_DF3_4F_NoiseLimitDetails.csv](#) (4.65kB). Límites normativos de ruido por zona y franja horaria. 8 columnas.
4. [04-Metadata_Indicator_API_ESP_DS2_en_csv_v2_17575.csv](#) (1.23MB). Datos socioeconómicos del Banco Mundial. Metadatos del indicador. 4 columnas.
5. [05-estaciones-acusticas.csv](#) (4.66kB). Información estructurada sobre estaciones de medición acústica. 8 columnas.
6. [06-200085-1-censo-locales.csv](#) (92.55MB). Censo parcial de locales y actividad económica. 46 columnas.
7. [06-200085-6-censo-locales.csv](#) (5.38MB). Complemento del censo de locales. 117 columnas.
8. [07-datos_calidad_vida_multi.xlsx](#) (97.56kB). Indicadores multidimensionales de calidad de vida. Organizado en 12 dimensiones y 20 columnas por dimensión.
9. [08-48130_salud_mental.csv](#) (50.83kB). Indicadores agregados de salud mental por zona. 6 columnas.
10. [09-31103_ADRH_distribucion_renta_hogares.csv](#) (451.29MB). Distribución de renta de los hogares. 8 columnas.

2.2. Kaggle – Repositorio con los datos procesados

Tras completar el proceso de limpieza, transformación y normalización de los datos originales, se ha generado un conjunto estructurado de datasets procesados, específicamente preparados para su análisis y posterior visualización. Estos ficheros representan la versión depurada, enriquecida y armonizada de las fuentes iniciales, garantizando consistencia entre variables y facilitando su integración en los cuadernos Jupyter del proyecto.

Dado su volumen y la necesidad de mantener un control de versiones estable, los datos procesados se alojan igualmente en Kaggle, donde pueden consultarse, descargarse y reutilizarse de forma reproducible. Este repositorio permite mantener la trazabilidad de las transformaciones realizadas y asegurar que el equipo docente pueda acceder a los ficheros finales sin restricciones de tamaño.

Repositorio de datos procesados (acceso público):

Kaggle Dataset (datos procesados):<https://www.kaggle.com/datasets/raquelahdo/ruido-datasets-procesados>

Tamaño total: 85.36 MB

Número de ficheros: 14

Los ficheros incluidos en la versión 4 del dataset con el que se han preparado los datos son los siguientes:

1. [df_01_Ruido_diario_acumulado_processed.csv](#) (47.2 MB) - 17 columnas
2. [df_01_daily_plot.csv](#) (2.87 MB) - 3 columnas
3. [df_01_daily_processed.csv](#) (548.02 kB) - 3 columnas
4. [df_02_contaminacion_acustica_processed.csv](#) (964.29 kB) - 17 columnas
5. [df_02_estacion_processed.csv](#) (936 B) - 2 columnas
6. [df_03_noise_limit_processed.csv](#) (6.08 kB) - 10 columnas
7. [df_05_estaciones-acusticas_processed.csv](#) (4.64 kB) - 11 columnas
8. [df_06_locales_terrazas_processed.csv](#) (33.54 MB) - 27 columnas
9. [df_07_Madrid-calidad-vida_processed.csv](#) (3.33 kB) - 19 columnas
10. [df_07_calidad-vida_processed.csv](#) (6.13 kB) - 18 columnas
11. [df_07_evolucion-calidad-vida_processed.csv](#) (2.77 kB) - 18 columnas
12. [df_07_evolucion-long-calidad-vida_processed.csv](#) (8.09 kB) - 4 columnas
13. [df_07_final-calidad-vida_processed.csv](#) (17.08 kB) - 7 columnas
14. [df_09_reduced_distribucion-renta_processed.csv](#) (186.82 kB) - 5 columnas

No se han generado datasets procesados correspondientes a los conjuntos de datos **04** (*04-Metadata_Indicator_API_ESP_DS2_en_csv_v2_17575.csv*, 1.23 MB) y **08** (*08-48130_salud_mental.csv*, 50.83 kB). Tras realizar el análisis exploratorio inicial, ambos fueron descartados por no aportar información relevante para los objetivos de la visualización planteada en este proyecto.

- El dataset 04 contiene exclusivamente metadatos de indicadores socioeconómicos del Banco Mundial y no incluye valores cuantitativos útiles para el análisis.
- Por su parte, el dataset 08 presenta indicadores agregados de salud mental con un nivel de detalle insuficiente y sin relación significativa con las variables principales del estudio sobre el impacto del ruido.

En consecuencia, ninguno de ellos será incorporado en el procesamiento ni en la construcción de los datos finales del proyecto.

3. Cuadernos Jupyter de preparación de datos

El proceso de preparación del conjunto de datos se ha llevado a cabo mediante una serie de **cuadernos Jupyter**, específicamente diseñados para realizar las tareas de limpieza, transformación y normalización necesarias antes del análisis y la posterior visualización. Estos cuadernos constituyen el núcleo operativo del proyecto, ya que documentan de forma reproducible cada una de las operaciones aplicadas a los datos, así como las decisiones metodológicas adoptadas en cada fase del procesamiento.

Todos los cuadernos están disponibles tanto en el **repositorio GitHub del proyecto**, [/notebooks](#), como en la **plataforma de la UNED**, facilitando así el acceso y la revisión por parte del equipo docente.

A continuación, se detallan los cuadernos incluidos en esta fase del trabajo:

- 01_Ruido_diario_acumulado.ipynb
- 02_Contaminacion_acustica.ipynb
- 03-Noise-limit.ipynb
- 05-estaciones-acusticas.ipynb
- 06-censo-locales.ipynb
- 07-calidad-vida.ipynb
- 08-salud-mental.ipynb
- 09-ADRH-distribucion renta.ipynb

En conjunto, estos cuadernos permiten documentar de manera precisa y transparente todo el flujo de preparación de datos, garantizando la trazabilidad completa del proceso y la capacidad de reproducir los resultados en cualquier entorno compatible.

4. Proceso seguido para la preparación de los datos para la visualización – Fase de Preprocesado y Procesado

El **proceso de preparación de los datos para la visualización** se ha desarrollado siguiendo un flujo metodológico estructurado, aplicado de forma independiente a cada uno de los conjuntos de datos mediante cuadernos Jupyter específicos de limpieza y normalización.

La **fase de preprocesado** comienza con la **obtención y carga de los datos** procedentes del repositorio de Kaggle, garantizando la trazabilidad de las versiones utilizadas. A continuación, se realiza un **análisis preliminar para identificar la estructura**, tipos de variables, presencia de valores nulos y posibles inconsistencias. Sobre esta base, se ha aplicado un **proceso general de limpieza**, que incluye corrección de formatos, estandarización de columnas, tratamiento de valores atípicos, verificación de coherencia temporal y depuración de registros irrelevantes o redundantes. Posteriormente, se **asegura la consistencia interna y entre datasets**, ajustando nomenclaturas, categorías y unidades para permitir su integración.

La **fase de procesado** se ha centrado en **generar datasets derivados optimizados** para su uso en Altair-Vega, garantizando compatibilidad semántica y estructural, reducción del peso de los ficheros, y mejora del rendimiento en operaciones que implican grandes volúmenes de filas.

Finalmente, se han generado **tablas limpias, normalizadas y coherentes**, listas para su explotación en la fase analítica y en la construcción de visualizaciones y correlaciones.

Este proceso asegura que todos los datos transformados cumplen los requisitos de calidad, reproducibilidad y eficiencia necesarios para la visualización final del proyecto.

DIAGRAMA GENERAL DE PREPROCESADO Y PROCESADO DE DATASETS

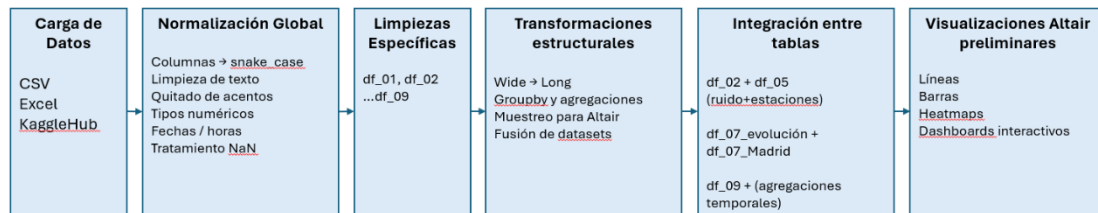


Figura 1: Diagrama de proceso de Preprocesado y procesamiento de los conjuntos de datos

4.1. Preparación del entorno

Se configura un entorno Python basado en Jupyter utilizando un conjunto reducido pero potente de librerías. **Pandas** y **NumPy** constituyen la base del tratamiento de datos. **KaggleHub** permite descargar automáticamente los ficheros directamente desde el repositorio Kaggle del proyecto, garantizando reproducibilidad. Para análisis estadístico se utilizan **scikit-learn** y **SciPy** (clustering jerárquico), que ayudan a detectar patrones, grupos y relaciones entre variables. **Matplotlib** y **Seaborn** complementan las visualizaciones interactivas proporcionando gráficos estáticos, como correlaciones, histogramas y heatmaps durante la fase exploratoria. Finalmente, **reverse_geocoder** se utiliza puntualmente para validar o enriquecer localizaciones a partir de coordenadas, y las **opciones globales de Pandas (pd.set_option)** permiten visualizar datasets grandes con facilidad durante la inspección inicial.

```

import pandas as pd
import reverse_geocoder as rg
import kagglehub
from kagglehub import KaggleDatasetAdapter
from sklearn.decomposition import PCA
from scipy.cluster import hierarchy
from scipy.cluster.hierarchy import fcluster
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
  
```

Configura Altair para trabajar de forma estable y eficiente con grandes volúmenes de datos. Se utiliza `alt.data_transformers.enable("default", max_rows=None)` que evita que

Altair limite automáticamente el número de filas y permite visualizar datasets extensos sin recortes, y `alt.renderers.enable("mimetype")` selecciona un renderer (motor de visualización) compatible con JupyterLab/Anaconda.

```
import altair as alt
# Desactivar vegafusion si está activado
alt.data_transformers.enable("default", max_rows=None)

# Renderer compatible con JupyterLab / Anaconda
alt.renderers.enable("mimetype")
```

4.2. Carga de datos

La carga de datos se realiza a partir de ficheros CSV, Excel y datasets alojados en KaggleHub. Para **CSV** se utiliza `pd.read_csv()`, ajustando la codificación según el origen del fichero (por ejemplo, `encoding="latin-1"` o `encoding="utf-8-sig"` para archivos con BOM). En el caso de **Excel**, se emplea `pd.read_excel(sheet_name=None)` cuando es necesario cargar múltiples hojas. Los datos almacenados en **KaggleHub** se descargan mediante `kagglehub.dataset_load()` o `dataset_download()` para obtener rutas locales.

Ejemplos de carga:

```
# CSV con codificación Latin-1
df = pd.read_csv("file.csv", encoding="latin-1")

# Excel con varias pestañas
df_excel = pd.read_excel("file.xlsx", sheet_name=None)

# Carga desde KaggleHub
file_path_05 = "05-estaciones-acusticas.csv"
df_05 = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "raquelahdo/ruido-datasets",
    file_path_05,
    pandas_kwargs={"encoding": "latin-1", "sep": ";"}
)
```

4.3. Renombrado y normalización de columnas

Se aplican las siguientes conversiones en todos los datasets cargados, con `rename()`:

- Convertir a minúsculas.
- Quitar acentos.
- Quitar espacios y caracteres especiales.
- Convertir a snake_case.

Por ejemplo:

```
df_05 = df_05.rename(columns={
    "ESTACIÓN": "estacion",
    "NOMBRE": "nombre",
    "UBICACIÓN": "ubicacion",
    "DISTRITO": "distrito",
    "BARRIO": "barrio",
    "LONGITUD": "longitud",
    "LATITUD": "latitud",
    "X": "utm_x",
```

```
"Y": "utm_y",
"ALTITUD": "altitud",
"FECHA ALTA": "fecha_alta"
})
```

4.4. Normalización de tipos de datos y optimización

Se aplican las siguientes conversiones en todos los datasets cargados:

- Convertir strings numéricos a float o int: `df[col] = pd.to_numeric(df[col], errors="coerce")`
- Convertir fechas: `df[col] = pd.to_datetime(df[col], errors="coerce")`
- Convertir horas, evitando campos relacionados con las horas que no funcionan en Altair: `df[col] = df[col].astype(str).str.strip()`
- Convertir booleanos: `df[col] = df[col].map({ "yes": True, "no": False })`
- Eliminación y tratamiento de valores nulos:
`df = df.replace({"": None, "nan": None, ".": None})`
`df = df.drop_duplicates()`
- Transformación de valores a Long:
`df_long = df.melt(id_vars=["dim"], var_name="año", value_name="valor")`
- Reducción de millones de filas para mejorar la eficiencia y rendimiento en Altair. Para gráficos de series temporales o histogramas. Se agrupan los datos con `groupby()` para reducir tamaño de datos, se muestrean o se agregan los datos antes de realizar los gráficos. Por ejemplo:
`df_temporal= df.groupby(["municipio","periodo"]).agg(renta_media= ("total","mean"))`

4.5. Fusión de Datasets

Los datasets relacionados se fusionan, con `merge()`, para facilitar la correlación de datos y cálculo de nuevas columnas y valores para el análisis. Por ejemplo:

```
df_final = df1.merge(df2, on=["clave"], how="left")
Datasets relacionados:
```

- Ruido + estaciones.
- Ruido + distritos.
- Locales y terrazas, se fusionan los 2 datasets.
- Calidad de vida + Evolución de calidad de vida.

4.6. Asegurar la compatibilidad de los datos con Altair-Vega

Se asegura la compatibilidad con Altair-Vega verificando:

- No hay columnas duplicadas.

```
df_07_final.columns[df_07_final.columns.duplicated()]
#Eliminamos columnas duplicadas
df_07_final = df_07_final.loc[:, ~df_07_final.columns.duplicated()]
```

-

- Se evita campos de hora con `datetime.time` convirtiendo a strings.

```
#Se construye la columna Fecha (de tipo datetime64).
df_01["fecha"] = pd.to_datetime(df_01[["year", "month", "day"]], errors="coerce")
#Columna: mes en formato nombre
df_01["mes_nombre"] = df_01["fecha"].dt.month_name(locale="es_ES.utf8")
#Columna: Año-mes
df_01["year_month"] = df_01["fecha"].dt.to_period("M").astype(str)
#Columna: semana del año
df_01["week"] = df_01["fecha"].dt.isocalendar().week
```

4.7. Detalle específico de Preprocesado y Procesado particular de datasets

Se detalla a continuación la fase de preprocesado y procesado cada dataset, incluido en los cuadernos Jupyter correspondiente para cada uno de los datasets.

4.6.1. df_01 – ruido diario

Tareas realizadas:

- Carga CSV.
- Limpieza de columnas.
- Conversión de variables de ruido a numéricas (laeq, ld, le, ln).
- Conversión de fecha a datetime.
- Normalización del nombre de estación.
- Eliminación de espacios y textos erróneos.

Detalle de código aplicado:

```
df_01["fecha"] = pd.to_datetime(df_01["fecha"])
df_01["laeq"] = pd.to_numeric(df_01["laeq"], errors="coerce")
df_01["estacion"] = df_01["estacion"].str.strip()
```

4.6.2. df_02 — Ruido mensual

Tareas realizadas:

- Misma limpieza que df_01
- Unificación con df_05 (estaciones) para añadir geolocalización.
- Tratamiento y eliminación de valores nulos.

4.6.3. df_03 — Noise Limit Details (fichero con umbral normativo europeo)

Tareas realizadas:

- Convertir caracteres BOM.
- Convertir columnas de categoría (source, status, areaType...).

- Convertir limitvalue a float.
- Normalizar indicadores (Lday, Levening, Lnight).
- Convertir limitvaluedefined a booleano.

Detalle del código aplicado:

```
df_03 = pd.read_csv("03-noiselimit.csv", encoding="utf-8-sig")
df_03["limitvalue"] = pd.to_numeric(df_03["limitvalue"], errors="coerce")
df_03["noiselevelindicator"] = df_03["noiselevelindicator"].str.lower()
```

4.6.4. df_04 – Datos socioeconómicos del Banco Mundial

Por falta de relevancia de los datos en el estudio, tras análisis preliminar, se ha decidido no utilizar el dataset df_04.

El dataset 04 contiene exclusivamente metadatos de indicadores socioeconómicos del Banco Mundial y no incluye valores cuantitativos útiles para el análisis.

4.6.5. df_05 — Estaciones de ruido

Tareas realizadas:

- Limpieza de coordenadas (latitud/longitud).
- Conversión UTM a float.
- Limpieza de textos de distrito/barrio.
- Unión posterior con df_02.

4.6.6. df_06 (df_061 y df_062) — Censo de locales y terrazas

Tareas realizadas:

- Nombres de columnas extremadamente largos, se han normalizado los nombres.
- Conversión de coordenadas.
- Conversión de campos de horarios a strings para evitar Altair errores.
- Conversión de booleanos.
- Se eliminan columnas sin información útil para el análisis.
- Unión de df_061 + df_062 por id_local.

Código aplicado:

```
df_062[col] = pd.to_datetime(df_062[col], format="%H:%M:%S", errors="coerce").dt.time
df_062[col] = df_062[col].astype(str) # evitar errores Altair
```

4.6.7. df_07 — Calidad de vida (Excel multipestaña)

Tareas realizadas:

- Carga multipestaña con sheet_name=None

- Selección de filas con datos relevantes para el estudio, con CCAA == "Madrid, Comunidad de".
- Extracción de pestaña "Evolución".
- Normalización de campo de wide a long.
- Unificación España vs Madrid por dimensión.

Código ejemplo utilizado:

```
df_07_raw = pd.read_excel(ruta, sheet_name=None)
df_07_Madrid = []
for hoja, df in df_07_raw.items():
    if "CCAA" in df.columns:
        df_mad = df[df["CCAA"]=="Madrid, Comunidad de"]
        df_mad["hoja"] = hoja
        df_07_Madrid.append(df_mad)
df_07_Madrid = pd.concat(df_07_Madrid)
```

Para unificación de los datos con melt():

```
df_07_evolucion_long = df_07_evolucion.melt(
    id_vars=["dim"], var_name="año", value_name="valor_evolucion"
)
```

4.6.8. df_08 - Salud mental

Por falta de relevancia de los datos en el estudio, tras análisis preliminar, se decide no utilizar el dataset df_08.

El dataset 08 presenta indicadores agregados de salud mental con un nivel de detalle insuficiente y sin relación significativa con las variables principales del estudio sobre el impacto del ruido.

4.6.9. df_09 — Renta por unidad de consumo (2.4M filas)

El dataset, df_09, presenta retos en la preparación de datos:

- Tamaño muy voluminoso, dispone de 2.4 millones de filas, y Altair no soporta datasets de estas dimensiones. Se agrupan los campos municipio/distrito/tramo de edad con **groupby()** para las visualizaciones.

```
df_temporal = df_09.groupby(["municipio_nombre", "periodo"]).agg(
    total_mean=("total", "mean")
)
```

- Incluye valores "." en columna total, que convertiremos a NaN.
 - Se eliminan los valores "." y NaN.

```
df_09 = df_09.dropna(subset=["total"])
```

- Convertir la columna "total" a valor numérico.

```
df_09["total"] = pd.to_numeric(df_09["total"], errors="coerce")
```